MTH 4300
Spring 2026

Programming Problem Set 5: Rule of Three - `dynamicarray.cpp`

Directions: Download the template files provided on Brightspace. Submit your source code via Gradescope, in `.cpp` format.

Submission Reminder: Submitting for feedback is optional. If you submit by the deadline and want feedback beyond the autograder, indicate that in your submission. You may resubmit unlimited times.
Important: Passing all test cases does not guarantee correctness. The autograder tests limited inputs. Test additional cases yourself.
Note: While learning, avoid IDE tools that autocomplete or generate code for you (e.g., Copilot, IntelliSense suggestions). Writing code yourself builds the understanding you'll need for exams.

Working with AI on This Assignment

This assignment has a structured AI component. Follow these steps in order.

Step 1: Write your code on your own first. Give each part a genuine attempt before using any AI. If your code compiles and runs but gives wrong answers, try tracing through it with a specific input on paper before moving on to Step 2.

Step 2: If you're stuck, use AI as a debugging tutor. Don't paste your code and ask "what's wrong." Instead, use a prompt like this:

> *"I'm writing a C++ class called DynamicArray that [describe what it should do]. Here is my code. I'm getting [wrong output / crash / compiler error]. Don't fix my code or show me corrected code. Instead, pick a specific line and ask me what I think a particular variable or expression evaluates to at that moment. Help me find the bug by making me trace through my own logic. Keep a back-and-forth conversation."*

If you use this step, save the conversation; you'll submit a link to it.

Step 3: After your code passes the autograder, do a code review conversation. The goal is to test whether you can explain what you wrote. Paste one of the prompts below along with your working code into a new AI conversation (ChatGPT, Claude, Copilot, etc.). The AI will quiz you; treat it like a study session. Dynamic memory management involves several important concepts (ownership, deep vs. shallow copy, resource cleanup).

Paste the prompt below along with your working code (dynamicarray.cpp) into a new AI conversation:

> *"Here is my working C++ code for a DynamicArray class (dynamicarray.cpp). Act as a technical interviewer. Ask me questions one at a time about my design and implementation choices. Start easier and build toward harder questions. If my answer is vague or wrong, don't correct me — ask a follow-up that helps me see the issue. Do 5–6 rounds. Then propose a small modification to my class and ask me to implement it. Critique my implementation."*

Submit a link to the conversation as a comment at the top of your `.cpp` file, along with any Step 2 conversation links.

Be sure to read the SPECIFICATIONS carefully! And write comments!

- No global variables should be used.
- The only header allowed is `iostream`.
- You cannot use vectors or any advanced data structure that can store an arbitrarily long collection of numbers.
- Note: Parts (a) and (b) have no user input. Part (c) involves user input.
- Remember to use `const` whenever appropriate.

Note: Each problem has test cases on Gradescope. All test cases are visible, so you can see exactly which cases your code passes or fails. Use this feedback to improve your solution. You may resubmit as many times as you like. Remember, these assignments are for practice; use them to prepare for exams!

5a) `DynamicArray` class - Part 1

Write a C++ class called `DynamicArray` that manages a dynamically allocated array of integers. The class should have the following specifications:

- **Private members:** `int* data` (pointer to the dynamically allocated array), `int size` (number of elements currently stored), and `int capacity` (total number of elements the array can hold before resizing).
- **Constructor** `DynamicArray(int cap = 10)` — allocates a dynamic array of the given capacity using `new`, and sets the size to 0. The parameter should default to 10 if not provided.
- **Destructor** — deallocates the dynamic memory using `delete[]`.

- `void push_back(int val)` — appends `val` to the end of the array; i.e. add the new element and increment `size`. If the array is full (i.e., `size == capacity`), the method should:
  1. Double the `capacity`.
  2. Allocate a new array with the doubled capacity.
  3. Copy all existing elements to the new array.
  4. Delete the old array.
  5. Update the `data` pointer.
  6. Add the new element and increment `size`.
- `int get(int index) const` — returns the element at position `index`. If `index` is out of bounds (less than 0 or greater than or equal to `size`), print `"Error: index out of bounds"` followed by a newline and return `-1`.
- `void set(int index, int val)` — sets the element at position `index` to `val`. If `index` is out of bounds, print `"Error: index out of bounds"` followed by a newline.
- `int getSize() const` — returns the current number of elements.
- `int getCapacity() const` — returns the current capacity.
- `void print() const` — prints all elements in the format `[1, 2, 3]` (comma-separated, enclosed in square brackets). An empty array prints `[]`. A newline should be printed at the end.

Your class should be split into two files:

- `DynamicArray.h` — the class declaration (member variables and method headers).
- `DynamicArray.cpp` — the method implementations. This file should `#include "DynamicArray.h"`.

Note: When separating your class into `.h` and `.cpp` files, default parameter values should appear only in the declaration (in the `.h` file), not in the implementation (in the `.cpp` file).

The template files include some tester code with expected output. Submit both `DynamicArray.h` and `DynamicArray.cpp` to Gradescope. Your files must be named exactly `DynamicArray.h` and `DynamicArray.cpp` (case-sensitive). Do not include a `main` function in either file.

To compile and test locally:

```
g++ DynamicArray.cpp main.cpp -o main
./main
```

5b) **Copy constructor and copy assignment operator**

Your `DynamicArray` class from Part (a) has a constructor and destructor, but it still relies on the compiler-generated default copy constructor and copy assignment operator. These perform a **shallow copy** — they copy the pointer itself, not the data it points to. This means two objects end up sharing the same dynamically allocated memory, which leads to problems like double-free errors and unintended modifications.

Add the following to your class:

- **Copy constructor** `DynamicArray(const DynamicArray& other)` — creates a new `DynamicArray` that is a **deep copy** of `other`. It should allocate its own memory and copy all elements from `other`.
- **Copy assignment operator** `DynamicArray& operator=(const DynamicArray& other)` — assigns the contents of `other` to the current object by performing a deep copy. This must:
  1. Check for **self-assignment** (i.e., `if (this == &other)`) and return early if so.
  2. Deallocate the current object's old memory.
  3. Allocate new memory and copy all elements from `other`.
  4. Update `size` and `capacity`.
  5. Return `*this` to allow chaining (e.g., `a = b = c`).

Together with the destructor from Part (a), these three components make up the **Rule of Three**: if a class manages a resource (like dynamic memory), it should explicitly define the destructor, copy constructor, and copy assignment operator.

Submit your updated `DynamicArray.h` and `DynamicArray.cpp` to the Part (b) submission area on Gradescope.

5c) **Client code + conceptual questions**

AI Code Review: After your code passes the autograder for Parts (a) and (b), complete the AI code review conversation described at the beginning of this assignment. Submit a link to the conversation as a comment at the top of your `main.cpp` file. See the "Working with AI on This Assignment" box on page 1 for the prompt and full instructions.

For this part, you will write a `main.cpp` file that uses the `DynamicArray` class. Your program should `#include "DynamicArray.h"` at the top. You must use the `DynamicArray` class — do not use `std::vector` or any other container.

Important: On Gradescope, your `main.cpp` will be compiled against a correct implementation of the `DynamicArray` class. You only need to submit `main.cpp` for this part. This means your Part (c) grade is independent of Parts (a) and (b).

To compile and test locally, place your `DynamicArray.h`, `DynamicArray.cpp`, and `main.cpp` in the same folder and run:

```
g++ DynamicArray.cpp main.cpp -o main
./main
```

Your program should do the following, in order. Read the instructions carefully and match the output format exactly. Do **NOT** print any input prompts (e.g., `"Enter a number:"`).

1. **Build from input:** Read an integer n from the user, then read n integers and push each one into a `DynamicArray` called `arr`. (I.e. the first input will indicate the number of integers that will be entered for the array.) Print the array using `print()`.
2. **Copy constructor:** Create a new `DynamicArray` called `copy1` that is a copy of `arr`, using the copy constructor (e.g., `DynamicArray copy1(arr);` or equivalently `DynamicArray copy1 = arr;`). Then modify `copy1` by setting its first element to `-999`. Print both `arr` and `copy1` to demonstrate that modifying the copy did not affect the original.

   Print:

   ```
   After modifying copy1:
   arr: [the original elements]
   copy1: [-999, rest of elements]
   ```
3. **Chained assignment:** Create two new `DynamicArray` objects `a` and `b` (using the default constructor). Use chained assignment: `a = b = arr;`. Print all three arrays. Then set the first element of `a` to `0` and the first element of `b` to `1`. Print all three again to show they are independent copies.

   Print:

   ```
   After chained assignment:
   arr: [the original elements]
   a: [the original elements]
   b: [the original elements]
   After modifying a and b:
   arr: [the original elements]
   a: [0, rest of elements]
   b: [1, rest of elements]
   ```
4. **Self-assignment:** Perform `arr = arr;` and then print `arr` to show it still works correctly.

   Print:

   ```
   After self-assignment:
   arr: [the original elements]
   ```

Answer the following questions **in comments** in your `main.cpp` file:

1. In step 2, you created `copy1` as a copy of `arr` (using either `DynamicArray copy1(arr);` or `DynamicArray copy1 = arr;`). What would happen if the class did not have a copy constructor and relied on the compiler-generated default? What specific error or problem would occur when one of the objects goes out of scope?
2. In step 3, the line `a = b = arr;` uses chained assignment. Explain why the copy assignment operator must return `*this` by reference for this to work.
3. In step 4, you did `arr = arr;`. What would go wrong if your copy assignment operator did not check for self-assignment? Trace through what would happen step by step.

MTH 4300 S26